Review 2

Three-address code for expressions and conditions

75% of the implementation

In this part

1. From checking to generating
2. Three-address code
3. Attributes
4. Backpatching helpers
5. Arithmetic to TAC
6. One condition, two jumps
7. Short-circuit ||
8. Short-circuit && and !
9. Review 2 output
10. Novelty

From checking to generating

REVIEW 1

Parser only

Reads tokens and says whether the syntax is valid.

Output: valid or invalid

REVIEW 2

Parser + translator

Emits code while parsing and leaves jump targets blank for later.

Output: numbered TAC

Quad + CodeGenerator

Attributes

Translation actions

Review 3 then uses these pieces to translate if and if-else statements.

Three-address code (TAC)

- At most 3 addresses: a result and two inputs.
- Temporaries t1, t2 hold partial results.
- Jumps: if ... goto and goto.

Source

```
x = a + b * 2;
```

TAC

```
t1 = b * 2
t2 = a + t1
x  = t2
```

Two jumps: `if a < b goto 4 | goto 7`

full_implementation.cpp: Quad, CodeGenerator::emit

```
struct Quad {
    int index;
    string op;      // + - * / = if goto print HALT
    string arg1;
    string arg2;
    string target;  // result or jump target
};

void emit(op, arg1, arg2, target) {
    code.push_back({nextQuad(), op,
                    arg1, arg2, target});
}
```

Attributes we carry along

ExprAttr

place

The name that holds the value: a, 5 or t1.

```
struct ExprAttr {
  string place;
  bool isConst;
  long long constVal;
};
```

BoolAttr

truelist, falselist

Jumps waiting for the true target and the false target.

```
struct BoolAttr {
  vector<int> truelist;
  vector<int> falselist;
};
```

StmtAttr

nextlist

Jumps waiting for whatever comes after the statement.

```
struct StmtAttr {
  vector<int> nextlist;
};
```

Backpatching: fill in later

The problem: when we write 'if a < b goto ?', we do not yet know where to go.

1. **makelist(i)**
   Start a list holding one instruction number.

2. **merge(p1, p2)**
   Join two lists into one.

3. **backpatch(p, target)**
   Fill every listed jump with the target.

full_implementation.cpp: CodeGenerator (bounds check omitted)

```
vector<int> makelist(int quadIndex) {
    return {quadIndex};
}

vector<int> merge(const vector<int>& l1,
                  const vector<int>& l2) {
    vector<int> res = l1;
    res.insert(res.end(), l2.begin(), l2.end());
    return res;
}

void backpatch(const vector<int>& list,
               int targetQuad) {
    for (int i : list)
        code[i].target = to_string(targetQuad);
}
```

Arithmetic to TAC

```
x = a + b * 2;
```

1. **parseTerm reads b * 2**
   emits `t1 = b * 2`

2. **parseExpr reads a + t1**
   emits `t2 = a + t1`

3. **parseAssignment stores it**
   emits `x = t2`

Multiplication is parsed deeper, so it runs first.

full_implementation.cpp: parseTerm (condensed)

```
// Term -> Factor ((* | /) Factor)*
ExprAttr f1 = parseFactor();
while (peek().type == TokenType::MUL ||
       peek().type == TokenType::DIV) {
    Token op = expect(peek().type);
    ExprAttr f2 = parseFactor();
    string temp = cg.newTemp();
    cg.emit(op.text, f1.place, f2.place, temp);
    f1 = ExprAttr(temp, false, 0);
}
```

One condition, two jumps

a < b

```
i:        if a < b goto ?
```

```
i + 1:    goto ?
```

truelist = { i }
falselist = { i + 1 }

A bare value such as if (x) is treated as x != 0.

full_implementation.cpp: parseBoolFactor (condition part)

```
ExprAttr e1 = parseExpr();
string relText = "!=";  string rightPlace = "0";
if (peek().type == TokenType::RELOP) {
    relText = expect(TokenType::RELOP).text;
    ExprAttr e2 = parseExpr();
    rightPlace = e2.place;
}
BoolAttr b;
b.truelist  = cg.makelist(cg.nextQuad());
b.falselist = cg.makelist(cg.nextQuad() + 1);
cg.emit("if", e1.place, relText + " " + rightPlace, "-1");
cg.emit("goto", "", "", "-1");
```

The target "-1" is a blank. backpatch() replaces it later.

Short-circuit OR (||)

Rule for B1 || B2

1. **If B1 is false, test B2**
   `backpatch(B1.falselist, M)`

2. **True if either is true**
   `truelist = merge(B1.true, B2.true)`

3. **False only if B2 is false**
   `falselist = B2.falselist`

full_implementation.cpp: parseBoolExpr

```
BoolAttr b1 = parseBoolTerm();
while (peek().type == TokenType::OR) {
    expect(TokenType::OR);
    int m = cg.nextQuad();    // M: B2 starts here
    BoolAttr b2 = parseBoolTerm();

    cg.backpatch(b1.falselist, m);
    b1.truelist  = cg.merge(b1.truelist,
                            b2.truelist);
    b1.falselist = b2.falselist;
}
```

In a < b || c == 10 the jump out of a < b goes to M, the first quad of c == 10.

Short-circuit AND and NOT

B1 && B2

- If B1 is true, test B2: backpatch(B1.truelist, M)
- False if either is false: merge the falselists
- True only if B2 is true

! B

Swap truelist and falselist. No new code is emitted.

full_implementation.cpp: parseBoolTerm, parseBoolFactor

```
// B1 && B2   (parseBoolTerm)
cg.backpatch(b1.truelist, m);
b1.falselist = cg.merge(b1.falselist,
                        b2.falselist);
b1.truelist  = b2.truelist;

// ! B   (parseBoolFactor)
BoolAttr b = parseBoolFactor();
return BoolAttr{b.falselist, b.truelist};
```

Short-circuit behaviour comes from where the jumps point, not from extra tests.

Review 2 output

Input program

```
a = 3; b = 5; c = 10;
x = a + b * 2;
if (a < b || c == 10)
    x = 1;
print x;
```

Arithmetic gives temporaries t1 and t2. The condition gives two jumps per comparison, with every target filled in.

```
================ THREE-ADDRESS CODE (TAC) ================
0:      a = 3
1:      b = 5
2:      c = 10
3:      t1 = b * 2
4:      t2 = a + t1
5:      x = t2
6:      if a < b goto 10
7:      goto 8
8:      if c == 10 goto 10
9:      goto 11
10:     x = 1
11:     print x
12:     HALT
==========================================================
```

Novelty

0

syntax trees stored

1

pass over the source

2

lists handle && and ||

- Compared with the classic tree route from the literature survey.
- Short-circuit logic is built into the jumps, with no pruning pass.
- The same truelist, falselist and nextlist carry the if-else translation in Review 3.